

SESIÓN

12

Acceso a datos con Entity Framework Core y LINQ + AJAX en ASP.NET Core

UNIDAD DE APRENDIZAJE 4

- Definición de Entity Framework Core (ORM)
- Generar el modelo desde una base de datos existente (database-first)
- Definición de LINQ y sus variantes
- Consultas con LINQ y procedimientos almacenados sobre EF Core
- AJAX moderno: actualizar parte de la página sin recargar

/ INTRODUCCIÓN

Hasta ahora la capa de datos hablaba con SQL Server a través de **ADO.NET**: abríamos conexiones, escribíamos comandos y leíamos **DataReaders** a mano. Funciona, pero implica mucho código repetitivo.

- **Entity Framework Core** es un ORM (Object-Relational Mapping): mapea las tablas a clases y los registros a objetos, para que trabajemos con datos como si fueran objetos de C#, sin escribir SQL a mano.
- **LINQ** es el lenguaje de consultas integrado en C#. Sobre EF Core nos deja consultar la base de datos con sintaxis de C#, con verificación en tiempo de compilación.
- **AJAX** permite que una página pida o envíe datos al servidor **en segundo plano** y actualice solo una parte, sin recargar todo. Hoy se hace con **fetch** y handlers de Razor Pages, no con controles de Web Forms.

Al terminar podrás: generar el modelo EF Core desde una BD existente, consultar con LINQ, ejecutar procedimientos almacenados desde EF Core y actualizar una página con AJAX moderno. **¡A trabajar!**

| / DEFINICIÓN DE ENTITY FRAMEWORK CORE

/ DEFINICIÓN DE ENTITY FRAMEWORK CORE

¿Qué problema resuelve un ORM?

- El programador piensa en **entidades del negocio** (Cliente, Factura, Producto) y sus relaciones. La base de datos piensa en **tablas, filas y columnas**. Son dos mundos distintos.
- Un **ORM** — Object (objeto) Relational (relacional) Mapping (mapeo) — es la capa que traduce automáticamente entre ambos: cada tabla pasa a ser una **clase**, cada fila una **instancia** y cada columna una **propiedad**.
- **Entity Framework Core** es el ORM oficial de .NET. Es la evolución del antiguo Entity Framework 6: multiplataforma, más ligero y el recomendado para proyectos sobre **.NET 10**.

En una frase: EF Core nos deja leer y grabar datos escribiendo C# en lugar de SQL, y se encarga por dentro de generar las consultas, manejar la conexión y las transacciones.

/ DEFINICIÓN DE ENTITY FRAMEWORK CORE

Del Entity Framework clásico (EF6) a EF Core

Tema	Antes (EF6 / .NET Framework)	Ahora (EF Core / .NET 10)
Modelo	Archivo visual <code>.edmx</code> (diseñador)	Clases C# + <code>DbContext</code> (sin diseñador)
Generación desde BD	Asistente "ADO.NET Entity Data Model"	Comando <code>Scaffold-DbContext</code>
Cadena de conexión	<code>App.config</code> / <code>Web.config</code>	<code>appsettings.json</code> + inyección
Instancia del modelo	<code>new MisEntities()</code>	Se inyecta el <code>DbContext</code> por constructor
Consultas	LINQ to Entities	LINQ to Entities (igual, pero asíncrono)
Plataforma	Solo Windows	Windows, Linux y macOS

Idea clave: el concepto es el mismo que viste con EF6 en el material original; cambia la forma de declararlo. Las consultas LINQ se escriben prácticamente igual.

/ DEFINICIÓN DE ENTITY FRAMEWORK CORE

Ventajas de usar EF Core

- **Menos código:** desaparecen las conexiones, comandos y lectores manuales de ADO.NET.
- **Tipado fuerte:** si te equivocas en el nombre de una columna, el error sale al **compilar**, no en tiempo de ejecución.
- **Seguimiento de cambios (Change Tracking):** EF Core sabe qué objetos cambiaron y arma el INSERT/UPDATE/DELETE por ti al llamar **SaveChanges()**.
- **Relaciones navegables:** desde una **Factura** accedes a **factura.Cliente** o a **factura.Detalles** como propiedades.
- **Independencia del motor:** el mismo código sirve para SQL Server, PostgreSQL o SQLite cambiando solo el proveedor.
- **Consultas asíncronas: ToListAsync()** libera el hilo mientras la BD responde, ideal para web.

| / GENERAR EL MODELO
DESDE UNA BASE DE DATOS

/ GENERAR EL MODELO DESDE UNA BASE DE DATOS

Dos formas de trabajar con EF Core

Database-First (la que usaremos)

Ya existe la base de datos (por ejemplo **VentasLeon**). EF Core lee su estructura y **genera** las clases y el DbContext automáticamente.

- Ideal cuando heredas una BD ya creada
- Se hace con el comando **Scaffold-DbContext**
- Equivale al asistente del **.edmx** de EF6

Code-First

Primero escribes las clases en C# y EF Core **crea** la base de datos a partir de ellas, usando migraciones.

- Ideal para proyectos nuevos desde cero
- Se usan **migraciones** (Add-Migration / Update-Database)
- Lo verás más adelante en el curso

En esta sesión usamos **database-first** sobre la BD **VentasLeon**, igual que el material clásico, pero con EF Core en lugar del diseñador **.edmx**.

/ GENERAR EL MODELO DESDE UNA BASE DE DATOS

Paso 1 · Instalar los paquetes y generar el modelo

En la capa de datos instalamos por NuGet los paquetes de EF Core para SQL Server y las herramientas:

```
Microsoft.EntityFrameworkCore.SqlServer
Microsoft.EntityFrameworkCore.Tools    // habilita los comandos de la consola
```

Luego, en la **Consola del Administrador de paquetes** (Package Manager Console), ejecutamos el comando que lee la BD y genera el modelo:

```
Scaffold-DbContext "Server=.;Database=VentasLeon;Trusted_Connection=True;TrustServerCertificate=True"
Microsoft.EntityFrameworkCore.SqlServer -OutputDir Modelos -Context VentasLeonContext
```

¿Qué hace? crea una clase por cada tabla (Cliente, Factura, ...) y un **VentasLeonContext** dentro de la carpeta **Modelos**. Es el reemplazo directo del antiguo asistente del .edmx.

/ GENERAR EL MODELO DESDE UNA BASE DE DATOS

El DbContext: la puerta de entrada a los datos

El **DbContext** representa la sesión con la base de datos. Cada tabla aparece como un **DbSet<T>**, que es la colección consultable:

```
public partial class VentasLeonContext : DbContext
{
    public DbSet<Factura> Facturas { get; set; }           // tabla Tb_Factura
    public DbSet<DetalleFactura> DetalleFacturas { get; set; }
    public DbSet<Cliente> Clientes { get; set; }
}
```

Comparación: donde EF6 hacía `new VentasLeonEntities()`, en EF Core no instanciamos el contexto a mano: lo **inyectamos** (siguiente diapositiva).

/ GENERAR EL MODELO DESDE UNA BASE DE DATOS

Paso 2 · Configurar la conexión y registrar el contexto

La cadena de conexión ya no va en Web.config: va en **appsettings.json**.

```
{
  "ConnectionStrings": {
    "VentasLeon": "Server=.;Database=VentasLeon;Trusted_Connection=True;TrustServerCertificate=True"
  }
}
```

Y en **Program.cs** registramos el contexto en el contenedor de dependencias:

```
builder.Services.AddDbContext<VentasLeonContext>(opciones =>
    opciones.UseSqlServer(builder.Configuration.GetConnectionString("VentasLeon")));
```

Inyección de dependencias: a partir de aquí cualquier página recibe el contexto listo en su constructor; no se crea con new.

| / DEFINICIÓN DE LINQ Y SUS VARIANTES

/ DEFINICIÓN DE LINQ Y SUS VARIANTES

¿Qué es LINQ?

- **LINQ** (Language Integrated Query) es un conjunto de operaciones de consulta integradas en C# para trabajar sobre distintas fuentes de datos: **objetos en memoria, bases de datos, XML**, etc.
- Su sintaxis se parece a SQL, pero vive **dentro del lenguaje**: el compilador la verifica y Visual Studio la autocompleta.
- Sobre EF Core se llama **LINQ to Entities**: EF Core **traduce** tu consulta LINQ a SQL y la ejecuta en el servidor de base de datos.

Existen dos formas equivalentes de escribir LINQ:

Sintaxis de consulta (query)

```
var q = from c in ctx.Clientes
        where c.Estado == 1
        select c;
```

Sintaxis de método (lambda)

```
var q = ctx.Clientes
        .Where(c => c.Estado == 1);
```

/ DEFINICIÓN DE LINQ Y SUS VARIANTES

Variantes de LINQ según la fuente de datos

Variante	Trabaja sobre	En este curso
LINQ to Objects	Colecciones en memoria (List, Array)	Filtrar listas ya cargadas
LINQ to Entities	Base de datos vía EF Core	El que usaremos hoy
LINQ to XML	Documentos XML	—
LINQ to DataSet	DataSets de ADO.NET clásico	—

Ejecución diferida (importante): la consulta LINQ **no** golpea la base de datos al declararla. Solo se ejecuta cuando recorres el resultado o llamas a `ToList()`, `ToListAsync()`, `First()`, `Sum()`, etc. Hasta entonces es solo una "receta".

**/ CONSULTAS CON LINQ Y
PROCEDIMIENTOS ALMACENADOS
| SOBRE EF CORE**

/ CONSULTAS CON LINQ Y PROCEDIMIENTOS ALMACENADOS

Ejemplo 1 · Deuda de un cliente (LINQ con join y Sum)

Suma los importes de las facturas **pendientes** (estado 1) de un cliente, uniendo cabecera y detalle:

```
public async Task<decimal> CalcularDeudaClienteAsync(string codigo)
{
    // consulta a la base de datos vía EF Core (LINQ to Entities)
    decimal deuda = await (from f in _ctx.Facturas
                           join d in _ctx.DetalleFacturas on f.NumFac equals d.NumFac
                           where f.CodCli == codigo && f.EstFac == 1
                           select (decimal)(d.CanVen * d.PreVen)).SumAsync();

    return deuda;
}
```

Observa: es casi idéntico al ejemplo de EF6 del material clásico. El cambio es que el contexto se llama **_ctx** (inyectado) y usamos **SumAsync()** para no bloquear el hilo.

/ CONSULTAS CON LINQ Y PROCEDIMIENTOS ALMACENADOS

Ejemplo 2 · Facturas de un cliente entre dos fechas

```
public async Task<List<Factura>> ListarFacturasAsync(string codigo, DateTime ini, DateTime fin)
{
    // el resultado se materializa solo al llamarToListAsync()
    return await _ctx.Facturas
        .Where(f => f.CodCli == codigo && f.FecFac >= ini && f.FecFac <= fin)
        .OrderBy(f => f.FecFac)
        .ToListAsync();
}
```

Sintaxis de método: aquí usamos Where + OrderBy. Es la forma más usada en el día a día con EF Core.

Traducción a SQL: EF Core convierte esto en un SELECT ... WHERE ... ORDER BY que se ejecuta en SQL Server, no en memoria.

/ CONSULTAS CON LINQ Y PROCEDIMIENTOS ALMACENADOS

Ejemplo 3 · Ejecutar un procedimiento almacenado

A veces el negocio ya tiene su lógica en un **procedimiento almacenado**. EF Core lo ejecuta con **FromSql** (devuelve filas) o **SqlQuery** (devuelve un valor escalar):

```
// el SP usp_ListarFacturasClienteFechas devuelve un conjunto de facturas
return await _ctx.Facturas
    .FromSql($"EXEC usp_ListarFacturasClienteFechas {codigo}, {ini}, {fin}")
    .ToListAsync();
```

```
// para un escalar (la deuda) se usa SqlQuery sobre Database
var deuda = await _ctx.Database
    .SqlQuery<decimal>($"SELECT dbo.fn_DeudaCliente({codigo}) AS Value")
    .FirstAsync();
```

Seguridad: usar interpolación `$"..."` con `FromSql` es seguro porque EF Core la convierte en **parámetros**, evitando inyección SQL. Nunca concatene cadenas con `+`.

| / AJAX EN ASP.NET CORE:
| ACTUALIZAR SIN RECARGAR

/ AJAX EN ASP.NET CORE

¿Qué es AJAX?

- **AJAX** = Asynchronous JavaScript And XML. Es la técnica que permite al navegador hablar con el servidor **en segundo plano** y actualizar solo una parte de la página.
- El usuario no ve recargas ni parpadeos: la experiencia es **fluida**, parecida a una app de escritorio.

Esquema tradicional

El navegador pide la página **completa**. El servidor la arma entera y la reenvía. Toda la pantalla se vuelve a dibujar.

Esquema con AJAX

El navegador pide **solo los datos que faltan**. El servidor responde un fragmento (JSON) y JavaScript actualiza únicamente esa zona.

Nota histórica: el nombre menciona XML, pero hoy casi siempre se intercambia **JSON**, que es más liviano.

/ AJAX EN ASP.NET CORE

Del AJAX Control Toolkit (Web Forms) al AJAX moderno

El material clásico instalaba el **AjaxControlToolkit**, una librería de controles de servidor para Web Forms. En ASP.NET Core esos controles ya no existen: todo se resuelve con HTML5 y JavaScript estándar, que funcionan en cualquier navegador.

Control clásico (Toolkit)	Para qué servía	Equivalente moderno
UpdatePanel	Actualizar parte de la página	fetch() + handler que devuelve JSON
ScriptManager	Coordinar el AJAX	Ya no se necesita
CalendarExtender	Selector de fecha	<input type="date">
MaskedEditExtender	Máscara de entrada	JavaScript / inputmode
ValidationCallout	Aviso de validación	Validación de cliente (jQuery / Bootstrap)
Confirmación	"¿Seguro de enviar?"	confirm() o modal de Bootstrap

/ AJAX EN ASP.NET CORE

AJAX en Razor Pages: handler + fetch

1) En el code-behind, un **handler** que devuelve JSON (su nombre empieza por **OnGet**):

```
// responde solo los datos del cliente; lo invoca el navegador con fetch
public async IActionResult OnGetBuscarCliente(string codigo)
{
    var c = await _ctx.Clientes.FirstOrDefaultAsync(x => x.CodCli == codigo);
    return new JsonResult(c == null ? null : new { c.CodCli, c.RazSocCli });
}
```

2) En la vista, JavaScript llama al handler y actualiza solo una parte:

```
// se llama al handler por su nombre: ?handler=BuscarCliente
const r = await fetch(`?handler=BuscarCliente&codigo=${codigo}`);
const data = await r.json();
document.getElementById("nombre").textContent = data ? data.razSocCli : "No existe";
```

Regla de oro (sigue vigente): la vista nunca habla directo con la base de datos. El fetch llama al handler, el handler usa la capa de datos (EF Core) y devuelve solo lo necesario.

| / LABORATORIO

/ LABORATORIO

Proyecto Web_Sesion12 · los tres ejemplos integrados

Construiremos un único sitio web (**VentasLeon**) con tres páginas, una por cada pilar de la sesión:

- **Ejemplo 1 — Consulta con LINQ:** calcular la deuda de un cliente y listar sus facturas entre dos fechas, usando EF Core + LINQ to Entities.
- **Ejemplo 2 — Procedimiento almacenado:** el mismo cálculo, pero ejecutando el SP **usp_ListarFacturasClienteFechas** desde EF Core con FromSql.
- **Ejemplo 3 — AJAX moderno:** buscar un cliente "en vivo" y actualizar el panel sin recargar (fetch + handler JSON), con selector de fecha, monto con máscara y confirmación antes de enviar.

Arquitectura: se mantienen las capas. La capa de datos expone métodos (LINQ y SP); las páginas solo los invocan. El manual paso a paso acompaña a este proyecto.

/ CONCLUSIONES

- **EF Core** es la versión moderna del ORM de .NET: maneja tablas como clases, registros como objetos y campos como propiedades, generando el SQL por nosotros.
- **LINQ to Entities** nos deja consultar la base de datos con C#, con verificación en compilación y ejecución diferida; y aun así podemos seguir usando **procedimientos almacenados** con FromSql.
- El **AJAX moderno** reemplaza al AjaxControlToolkit: con **fetch** y handlers que devuelven JSON actualizamos parte de la página sin recargar, manteniendo la regla de que la vista nunca toca la base de datos.
- El concepto es el mismo del material clásico; lo que cambia es que ahora es multiplataforma, más simple y aprovecha todo el modelo de .NET 10.

/ BIBLIOGRAFÍA

Documentación y recursos de consulta

- **MICROSOFT — Entity Framework Core.** Documentación oficial. Recuperado de <https://learn.microsoft.com/ef/core/>
- **MICROSOFT — Scaffolding (Reverse Engineering).** Generar el modelo desde una BD existente. Recuperado de <https://learn.microsoft.com/ef/core/managing-schemas/scaffolding/>
- **MICROSOFT — Consultas con LINQ en EF Core.** Recuperado de <https://learn.microsoft.com/ef/core/querying/>
- **MICROSOFT — Consultas SQL sin formato (FromSql).** Recuperado de <https://learn.microsoft.com/ef/core/querying/sql-queries>
- **MDN WEB DOCS — Uso de fetch.** Recuperado de https://developer.mozilla.org/es/docs/Web/API/Fetch_API/Using_Fetch
- **MICROSOFT — Razor Pages: handlers de página.** Recuperado de <https://learn.microsoft.com/aspnet/core/razor-pages/>